ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»
Факультет информатики, математики и компьютерных наук

Цапаев Данил Вадимович

СОЗДАНИЕ ПЛАТФОРМЫ ДЛЯ АНАЛИТИКИ ФИНАНСОВЫХ РЕЗУЛЬТАТОВ ПУБЛИЧНЫХ КОМПАНИЙ

Выпускная квалификационная работа -
БАКАЛАВРСКАЯ РАБОТА по направлению подготовки
Программная инженерия

образовательная программа

«09.03.04 Программная инженерия»

Рецензент
_____

приглашенный

преподаватель

Е.А. Лупанова

Руководитель старший преподаватель
_____

Н. С. Окуньков

Нижний Новгород, 2025

Введение

В условиях активной цифровизации экономических процессов и экспоненциального роста объёмов финансовой информации существенно возрастает значимость эффективных инструментов сбора, обработки и анализа сведений о финансовом положении публичных компаний. Для инвесторов, профильных специалистов в области инвестирования и представителей частного сектора критически важным становится не просто владение набором ключевых показателей отчётности, но и возможность их анализа, представленного в удобной, структурированной форме.

Традиционный подход, предполагающий поиск необходимых данных в различных источниках, влечёт за собой значительные временные затраты. При этом получаемая информация зачастую носит неполноценный характер, что не позволяет сформировать полное представление о реальном финансовом состоянии организации. В этой связи разработка веб-платформы для анализа показателей деятельности публичных компаний представляется актуальной и практически значимой задачей.

Предлагаемая система призвана объединить в едином информационном пространстве ключевые метрики финансовой отчётности, актуальные биржевые котировки и инструменты визуализации данных. Такая интеграция предоставит пользователю возможность проводить сравнительный анализ важнейших финансово-экономических показателей — в том числе динамики выручки и прибыли, уровня рентабельности, долговой нагрузки и других релевантных индикаторов — что послужит основой для принятия обоснованных инвестиционных решений.

Цель данного проекта — разработать программный инструмент, обеспечивающий полный анализ деятельности эмитентов и мониторинг инвестиционной активности в режиме реального времени. Создание данной платформы позволит существенно упростить доступ к финансовой аналитике, минимизировать рутинные операции по сбору и обработке данных, а также повысить качество управленческих решений за счёт опоры на структурированную, систематизированную информацию.

Глава 1. Постановка задачи и разработка требований

1.1 Задачи и цели

Я разрабатываю веб-платформу, которая станет единым рабочим пространством для инвестора. Ее главная задача — помочь пользователю глубоко анализировать финансовое состояние публичных компаний. Система объединит в себе ключевые направления: наглядную аналитику (выручка, прибыль, рентабельность, долги и дивиденды),Графики и основные прогнозы аналитиков.

Чтобы проект стал полноценным рабочим инструментом, мне предстоит пройти через несколько этапов:

- Сначала я изучу предметную область: разберусь, какие именно показатели особенно важны для инвесторов и как лучше структурировать данные о биржевых сделках. На основе этого я сформулирую четкие требования к системе — от поиска компаний до автоматического расчета доходности портфеля.
- Затем начнется техническая часть. Я спроектирую архитектуру приложения и создам надежную базу данных, которая сможет быстро обрабатывать большие объемы информации о рыночных данных.
- Разработка будет разделена на два модуля. Первый — Работа с API.Второй - рассчет коэфициентов и построение графиков.
- Особое внимание я уделю интерфейсу. Он должен быть интуитивным, чтобы поиск данных и ввод сделок не превращались в рутину. После написания кода я проведу тщательное тестирование: проверю точность всех расчетов, работу фильтров и устойчивость системы к ошибкам.
- В финале я оценю, насколько платформа справляется со своей задачей и куда ее развивать дальше. В будущем это могут быть автоматические выгрузки данных или прямая интеграция с брокерами. В итоге получится готовый продукт, который упростит анализ рынка и сделает контроль за инвестициями удобнее.

1.2 Требования к подсистеме

Основным этапом в выполнении работы стал процесс разработки требований. Этот шаг позволяет спланировать работу, оценить объем разработки и определить функционал готового продукта.

1.2.1 Требования к дизайн-макетам

При разработке дизайн-макетов платформы были сформированы требования, направленные на обеспечение удобства использования, визуальной целостности интерфейса и корректного отображения системы на различных устройствах.

Понятная структура и удобство навигации

Хороший дизайн — это прежде всего удобство.Информация о компании, финансовые графики должны быть под рукой.

Сложную структуру можно оставить только для служебных или административных разделов, где спешка не так критична.

Удобно, если на экране всплывают короткие уведомления: зеленые для успеха, желтые для предупреждений и красные для ошибок. Это избавляет от лишних вопросов и делает работу с системой интуитивной.

Единый визуальный стиль платформы

Платформа должна выглядеть как единое целое: стиль должен сразу считываться как инструмент для финансовой аналитики и инвестиционного мониторинга. Это значит, что всё — от шрифтов и кнопок до сложных таблиц и графиков — должно работать в рамках одной дизайн-системы.

Визуальный стиль, включая логотип, должен транслировать суть сервиса: работу с рыночными данными и инвестиционными расчетами. Чтобы подчеркнуть деловой характер системы, лучше придерживаться палитры, которая ассоциируется со стабильностью и цифровой средой.

Адаптация интерфейса под различные устройства

Макеты должны работать везде: и на маленьком смартфоне, и на огромном мониторе. Главное, чтобы интерфейс оставался читаемым и удобным, какой бы экран пользователь ни выбрал. Но дело не только в том, чтобы просто сжать или растянуть блоки. Нужно продумать саму логику: как перестроится меню, как изменятся таблицы, карточки или графики. Адаптивность — это не просто масштабирование, а умная перестройка контента под доступное пространство.

Использование компонентного подхода в дизайне

Важно продумать каждый компонент : как он выглядит в обычном состоянии, при наведении, в момент загрузки или если произошла ошибка. Если заранее предусмотреть все варианты — активный, отключенный или ошибочный — интерфейс станет предсказуемым, а масштабировать платформу будет гораздо проще. Главная цель здесь — создать универсальные инструменты, которые можно просто брать и вставлять в любой раздел системы, не тратя время на отрисовку новых решений.

Информативность экранов и логика взаимодействия

На каждом экране должно быть ровно столько информации, сколько нужно для дела.Важна и сама логика: человек должен с первого взгляда понимать, куда нажать, как отфильтровать данные, переключить период или открыть детали по компании. Чем меньше пользователю приходится задумываться над тем, как работает система, тем проще и полезнее она становится.

1.2.2 Требования к интерфейсной части веб-приложения

К интерфейсной части веб-приложения предъявляется ряд требований, обеспечивающих удобство разработки, поддерживаемость проекта, качество пользовательского взаимодействия и стабильность работы системы в целом.

Архитектура клиентской части

Интерфейс лучше строить на компонентном подходе. Так я разделю логику на понятные уровни: от простых базовых элементов до полноценных страниц. Каждый компонент будет отвечать за свою задачу, а мне это даст возможность легко переиспользовать код и масштабировать проект.

С папками тоже стоит не экспериментировать, а придерживаться строгой логики. Я разделю всё по зонам ответственности — так будет проще ориентироваться, когда проект разрастется. То же самое касается и маршрутов: URL должны быть предсказуемыми и отражать суть страниц. Если пользователь переходит в аналитику компаний, его карточку эмитента или историю сделок, адрес в строке браузера должен быть максимально понятным и логичным.

Типизация и контроль данных

Серверная часть написана на Python с использованием строгой типизации — это помогает избежать ошибок и делает код понятным при поддержке.Типизировать нужно всё: классы, функции, утилиты, сервисы, структуры данных и аргументы методов.

Отдельное внимание уделяется данным из API. Поскольку проект связан с финансами, любая неточность в структуре запросов или ответах сервера критична.Это позволяет выявлять ошибки на ранних этапах — ещё до попадания данных в бизнес-логику.

Маршрутизация и разграничение доступа

Нужно настроить проверку прав доступа так, чтобы система перехватывала пользователя при попытке перехода на закрытые страницы. Если человек пытается зайти в личный кабинет приложение должно сначала проверить его токен и роль через промежуточный слой маршрутов.

Управление состоянием приложения

Если данные нужны в разных частях интерфейса, лучше не создавать копии, а использовать центральное хранилище. В него стоит выносить всё, что должно быть под рукой: профиль пользователя, настройки приложения и ответы от API, которые запрашивают сразу несколько модулей.Лучше собрать повторяющиеся операции в отдельные функции или независимые модули. Так проект станет чище, код перестанет дублироваться, а компоненты не будут слишком сильно зависеть друг от друга. В итоге поддерживать и развивать приложение будет гораздо проще.

1.2.3 Нефункциональные требования

- **Скорость и производительность**

Я ставлю задачу: основные страницы должны открываться быстрее чем за 2 секунды, а любые манипуляции с данными (фильтры, сортировки, графики) — не дольше секунды. Чтобы этого добиться, нужно оптимизировать запросы к серверу и не перегружать загрузку данных.

- **Безопасность**

Защита данных и разграничение доступа — приоритет.Защищаю интерфейс от XSS-атак и на каждом закрытом маршруте обязательно проверяю права пользователя.

- **Удобство (UX)**

Платформа должна быть интуитивной. Интерфейс должен быть единообразным, а система — всегда давать обратную связь: уведомлять об успехе, ошибках. В формах ввода обязательно добавлю подсказки и валидацию, чтобы пользователь видел, если что-то заполнил не так.

- **Масштабируемость и рост**

Система должна расти вместе с бизнесом без переписывания архитектуры с нуля. Я закладываю возможность легко подключать новые модули. Чтобы большие объемы данных не ломали систему.

- **Совместимость**

Продукт должен одинаково хорошо работать везде: от смартфонов и планшетов до десктопов, на любых разрешениях экрана. Поддерживаем актуальные версии Chrome, Firefox, Edge и Safari.

- **Поддержка и обслуживание**

Я сдела упор на чистоту кода и понятную структуру проекта. Также обязательно настрою логирование ошибок на стороне интерфейса — это поможет быстро находить и устранять проблемы.

1.2.4 Функциональные требования:

Основные требования к моей веб-платформе для анализа финансов публичных компаний.

Регистрация и личный кабинет

Мне нужно, чтобы пользователи могли создавать аккаунты и безопасно хранить свои данные. При регистрации система должна проверять корректность почты и заполненность полей, а при входе — пускать пользователя в закрытые разделы. У каждого клиента должен быть личный кабинет с настройками и возможностью выйти из системы. Если пользователь ошибся при вводе пароля, нужно показать простое и понятное предупреждение.

Работа с данными компаний

Главная задача здесь — дать удобный доступ к информации о компаниях. В системе будет общий список, где любую организацию можно быстро найти по названию или тикеру. Для каждой компании я сделаю отдельную карточку: сначала общая справка, затем ключевые финансовые показатели. Информация должна быть структурирована так, чтобы можно было легко перейти от общего обзора к деталям. Если каких-то данных по компании не хватает, система должна об этом сообщить.

Анализ финансовых результатов

Чтобы пользователи могли оценивать динамику бизнеса, платформа должна показывать финансовые метрики за несколько периодов. Я добавлю возможность отслеживать выручку, прибыль, долги и рентабельность. Для наглядности данные будут

представлены и в таблицах, и в виде графиков — так тренды роста или падения считываются гораздо быстрее. Также система должна сама рассчитывать основные финансовые коэффициенты, помогая пользователю сразу видеть важные изменения в показателях.

Визуализация данных

В интерфейсе я использую цветовое выделение: так сразу видно, где идет рост, а где падение ключевых показателей.

Фильтрация и сортировка

Работать с большими массивами данных проще, если использовать фильтры. Вы можете сортировать графики,выбрать период отображения в кварталах ли годах.

Уведомления и обратная связь

Система подтверждает каждое действие.Если что-то пойдет не так, вы получите понятное сообщение об ошибке без лишнего технического жаргона. Если данные введены некорректно, система подскажет, где именно нужно внести исправления. Все уведомления заметны, но не перегружают интерфейс, обеспечивая прозрачный и комфортный процесс работы.

Глава 2. Обзор существующих решений

Перед разработкой собственной веб-платформы для анализа финансовых результатов публичных компаний необходимо рассмотреть уже существующие цифровые решения, используемые частными инвесторами и аналитиками. Такой обзор позволяет определить сильные и слабые стороны аналогов, выявить наиболее востребованные функции, а также понять, какие подходы и интерфейсные решения целесообразно использовать в разрабатываемой системе.

Современный рынок финансовых сервисов предлагает большое количество платформ, ориентированных на отображение рыночных данных, фундаментальный анализ компаний, ведение инвестиционного портфеля и визуализацию ключевых показателей. Однако большинство таких решений либо ориентировано только на один аспект работы с данными, либо обладает избыточной сложностью, либо требует платной подписки для получения доступа к важным возможностям. В связи с этим актуальной задачей является создание веб-платформы, объединяющей основные функции анализа и учета пользовательских данных в едином интерфейсе.

2.1. TradingView

TradingView является одной из наиболее популярных платформ для анализа финансовых рынков. Основное внимание в данном сервисе уделяется визуализации рыночной информации, работе с графиками и техническому анализу. Пользователю предоставляется доступ к широкому набору интерактивных графиков, инструментов рисования, индикаторов, а также к информации по акциям, валютам, индексам и другим активам.

Сильной стороной TradingView является удобный и современный интерфейс, а также высокая наглядность представления данных. Пользователь может быстро получить доступ к котировкам, динамике цен и ключевым рыночным метрикам. Платформа хорошо адаптирована как для профессиональных аналитиков, так и для частных инвесторов. Вместе с тем фундаментальный анализ финансовой отчетности в TradingView представлен ограниченно по сравнению со специализированными аналитическими сервисами. Кроме того, часть функций недоступна без оформления платной подписки.

Решения, которые можно взять:

- удобная визуализация данных с помощью интерактивных графиков;
- современный и лаконичный интерфейс;
- структурированное отображение информации об активе;
- использование фильтров и переключателей временных интервалов;
- выделение ключевых показателей в отдельных информативных блоках.

2.2. Yahoo Finance

Yahoo Finance — это широко известный онлайн-сервис, предоставляющий финансовую информацию по публичным компаниям, биржевым инструментам и рыночным индексам. Платформа предлагает пользователям данные о стоимости акций, рыночной капитализации, отчетности, новостях и некоторых аналитических показателях. Одним из преимуществ сервиса является открытый доступ к большому объему информации и простота использования.

Платформа позволяет быстро находить компании по названию или тикеру, просматривать исторические данные и отслеживать базовые рыночные показатели. Также Yahoo Finance поддерживает создание пользовательских списков наблюдения. Однако аналитические возможности системы ограничены, а глубина проработки финансовой отчетности и пользовательских инструментов уступает специализированным решениям.

Решения, которые можно взять:

- быстрый поиск компаний по тикеру и названию;
- карточки компаний с краткой сводной информацией;
- разделение данных на вкладки: обзор, финансовые показатели, новости, история;
- удобная структура пользовательского наблюдения за активами;
- простая и понятная навигация между разделами.

2.3. Investing.com

Investing.com представляет собой крупную финансовую платформу, сочетающую рыночные данные, новости, экономический календарь и базовые аналитические инструменты. Сервис ориентирован на широкий круг пользователей и предоставляет доступ к информации по акциям, валютам, сырьевым товарам, облигациям и индексам. Одной из сильных сторон платформы является большое количество доступных источников информации и высокая информативность страниц активов.

Пользователь может просматривать котировки, динамику, новости, аналитику и прогнозы, а также пользоваться календарем экономических событий. Вместе с тем интерфейс сервиса местами перегружен информацией, что может затруднять восприятие данных, особенно для начинающих пользователей. Кроме того, большое количество второстепенного контента может отвлекать от основной аналитической задачи.

Решения, которые можно взять:

- объединение аналитики, новостей и рыночных данных в одной системе;
- использование сводных блоков с ключевой информацией;
- отображение связанных событий, которые могут влиять на финансовые показатели;
- возможность быстрого перехода между различными категориями данных;
- представление исторической информации в удобной табличной форме.

2.4. Simply Wall St

Simply Wall St — это аналитическая платформа, ориентированная на наглядное представление фундаментальных данных о компаниях. Отличительной особенностью сервиса является активное использование визуализации: финансовая информация, риски, оценка стоимости компании и перспективы роста отображаются в виде диаграмм, карточек и инфографики. Такой подход делает сложные финансовые данные более понятными для широкой аудитории.

Платформа хорошо подходит для первичного анализа компаний, так как помогает быстро выделять сильные и слабые стороны бизнеса. Однако часть расчетов и выводов сервиса представляется в упрощенном виде, что может быть недостаточно для пользователей, которым требуется детальный доступ к исходным данным. Кроме того, многие функции ограничены платным доступом.

Решения, которые можно взять:

- акцент на визуальном представлении сложной финансовой информации;
- использование цветовых маркеров для оценки состояния показателей;
- представление аналитики в виде карточек, диаграмм и кратких выводов;
- упрощение восприятия финансовых коэффициентов для пользователя;
- выделение рисков и преимуществ компании в отдельные блоки.

2.5. Tinkoff Инвестиции / брокерские платформы

Брокерские платформы, такие как Тинькофф Инвестиции, Альфа-Инвестиции и другие, ориентированы не только на предоставление рыночной информации, но и на совершение реальных операций с активами. Для пользователя они удобны тем, что сочетают информацию по инструментам, портфель, историю сделок и отслеживание финансового результата. Как правило, такие сервисы обладают простым и дружелюбным интерфейсом, хорошо адаптированным под массового пользователя.

Вместе с тем аналитика фундаментальных показателей компании в таких системах обычно не является основной задачей и представлена в сокращенном виде. Основной фокус сделан на удобстве торговли, состоянии портфеля и текущем результате инвестора.

Решения, которые можно взять:

1. удобное отображение ключевых показателей

- адаптивный интерфейс для мобильных и настольных устройств.

2.6. Сравнительный анализ существующих решений

Рассмотренные платформы демонстрируют различные подходы к представлению финансовой информации и взаимодействию с пользователем. Одни решения делают акцент на техническом анализе и графиках, другие — на фундаментальной отчетности, третьи — на удобстве управления личным портфелем. Это показывает, что универсального решения, в полной мере на практике немного.

На основании проведенного обзора можно сделать вывод, что для разрабатываемой платформы целесообразно объединить наиболее удачные элементы существующих систем, адаптировав их под цели проекта.

Решения, которые можно взять в собственную разработку:

- из TradingView — интерактивные графики и удобную визуализацию динамики;
- из Yahoo Finance — простой поиск компаний и логичную структуру карточки компании;
- из Investing.com — совмещение данных, новостей и аналитики в одном интерфейсе;
- из Simply Wall St — наглядное представление финансовых коэффициентов и рисков;

2.7. Вывод по главе

Проведенный обзор показал, что существующие решения обладают рядом полезных функций и удачных интерфейсных подходов, которые могут быть использованы при проектировании собственной веб-платформы. Наиболее ценными являются механизмы быстрого поиска компаний, наглядная визуализация финансовых данных, отображение ключевых коэффициентов.

Глава 3. Выбор инструментальных средств и технологий

В настоящее время существует большое количество технологий и программных средств, применяемых при разработке веб-приложений. Каждое из них обладает собственными преимуществами, ограничениями и областями наиболее эффективного применения. Выбор инструментов для разработки должен основываться не только на их популярности, но и на соответствии целям проекта, удобстве сопровождения, скорости разработки и возможностях масштабирования.

В данном разделе обосновывается выбор технологий, использованных при разработке веб-платформы для анализа финансовых результатов публичных компаний и учета пользовательских инвестиционных сделок.

- **Django (6.1.1)**

Для реализации серверной части приложения выбран фреймворк Django версии 6.1.1. Он предоставляет полноценную экосистему для быстрой разработки надёжных веб-приложений: встроенные механизмы аутентификации, администрирования, работы с формами, валидации данных и управления сессиями. Для финансового проекта это критично: Django помогает минимизировать риски за счёт устоявшихся паттернов безопасности и строгой структуры приложения.

Единая архитектура Django позволяет держать бизнес-логику, маршрутизацию и обработку данных в понятной и предсказуемой структуре, что упрощает поддержку и рефакторинг кода по мере роста проекта.

- **Python**

Язык Python выбран как основа стека разработки. Его читаемость, богатая стандартная библиотека и обширная экосистема делают его отличным выбором для финансовых систем, где важна точность вычислений, прозрачность логики и скорость прототипирования. Python позволяет сосредоточиться на предметной области, а не на инфраструктурных деталях.

PostgreSQL

В качестве системы управления базами данных была выбрана PostgreSQL.

PostgreSQL является одной из наиболее надежных и функциональных реляционных СУБД. Она хорошо подходит для проектов, в которых необходимо хранить структурированные данные, связанные между собой отношениями. В рамках данного проекта база данных используется для хранения информации о пользователях, сделках, параметрах портфеля и других сущностях системы.

Реляционная модель данных удобна для реализации запросов, связанных с выборкой, фильтрацией, сортировкой и агрегацией информации. Это особенно важно для аналитической платформы, где требуется работать с финансовыми данными и историей пользовательских операций.

Дополнительно преимуществом PostgreSQL является устойчивость, поддержка сложных запросов и хорошая совместимость с современными ORM-инструментами.

Django ORM

Для взаимодействия с базой данных применяется встроенная ORM-система Django. Она позволяет описывать модели данных в виде Python-классов и работать с ними через понятный API, абстрагируясь от низкоуровневых SQL-запросов.

ORM обеспечивает типобезопасность на уровне моделей, автоматическую генерацию миграций и защиту от SQL-инъекций. Для финансового приложения это важно: структура данных остаётся согласованной, а изменения схемы базы данных управляются контролируемо.

- **Встроенные механизмы аутентификации и сессий Django**

Для реализации аутентификации и управления пользовательскими сессиями используются встроенные средства Django:

- **Система аутентификации** (модули auth и contrib.auth) — предоставляет готовые модели пользователей, механизмы регистрации, входа, смены пароля и проверки прав доступа.
- **SessionMiddleware** — обеспечивает хранение пользовательской сессии на сервере, позволяя сохранять состояние между запросами без передачи чувствительных данных клиенту.

Такой подход снижает риски ошибок при реализации авторизации и соответствует лучшим практикам безопасности (например, хранение токенов сессии на стороне сервера, защита CSRF-токенами и т. д.).

- ## Шаблонизатор Django Templates

Для рендеринга HTML-страниц используется встроенный шаблонизатор Django. Он позволяет разделять логику представления и бизнес-логику: шаблоны содержат разметку и простые инструкции по выводу данных, а вся сложная логика остаётся в представлениях (views).

Шаблонизатор поддерживает наследование шаблонов, кеширование фрагментов и безопасное экранирование вывода — это помогает избежать XSS-уязвимостей и поддерживать единообразный дизайн страниц.

- ## REST-подход в маршрутизации и обработке запросов

Взаимодействие компонентов приложения построено на принципах REST. В Django это реализуется через URL-маршруты (django.urls.path) и представления, которые обрабатывают HTTP-запросы (GET, POST, PUT, DELETE) и возвращают соответствующие ответы.

REST-подход удобен для чёткого разделения ответственности: URL определяют ресурсы, представления — логику их обработки, а шаблоны или JSON-ответы — формат представления данных. Такой стиль особенно полезен, когда в будущем планируется добавить API для интеграции с другими сервисами или мобильными клиентами.

- ## Управление конфигурацией и секретами

Конфигурация проекта вынесена в отдельный модуль настроек (settings.py), где параметры разделены на базовые и зависящие от окружения. Для загрузки переменных

окружения используется библиотека python-dotenv, а сами секреты (пароли, ключи, настройки БД) хранятся в файле .env.

Это позволяет безопасно управлять настройками в разных окружениях (разработка, тестирование, продакшн) и избегать жёстко прописанных значений в коде.

Tinkoff Invest API

В качестве внешнего источника финансовых данных был использован Tinkoff Invest API. Данный API предоставляет доступ к информации о финансовых инструментах и рыночных данных по протоколу HTTPS REST. Использование внешнего API позволяет получать актуальные данные, необходимые для отображения информации о компаниях, котировках и связанных аналитических показателях.

Подключение Tinkoff Invest API делает платформу более практической, поскольку данные поступают из реального внешнего источника, а не задаются вручную. Это повышает прикладную ценность системы и приближает ее к реальным условиям эксплуатации.

Figma

Для проектирования интерфейсов и предварительной подготовки визуальных решений использовалась Figma.

Данный инструмент позволяет разрабатывать макеты экранов, проектировать пользовательские сценарии и согласовывать внешний вид приложения до этапа программной реализации. Использование Figma способствует более структурированной разработке интерфейса и снижает количество доработок на поздних этапах проекта.

GitHub

Для хранения исходного кода использовался GitHub.

Размещение репозитория на GitHub обеспечивает централизованное хранение кода и удобство резервного копирования.

Применение этих инструментов особенно важно в процессе поэтапной разработки, тестирования и сопровождения приложения.

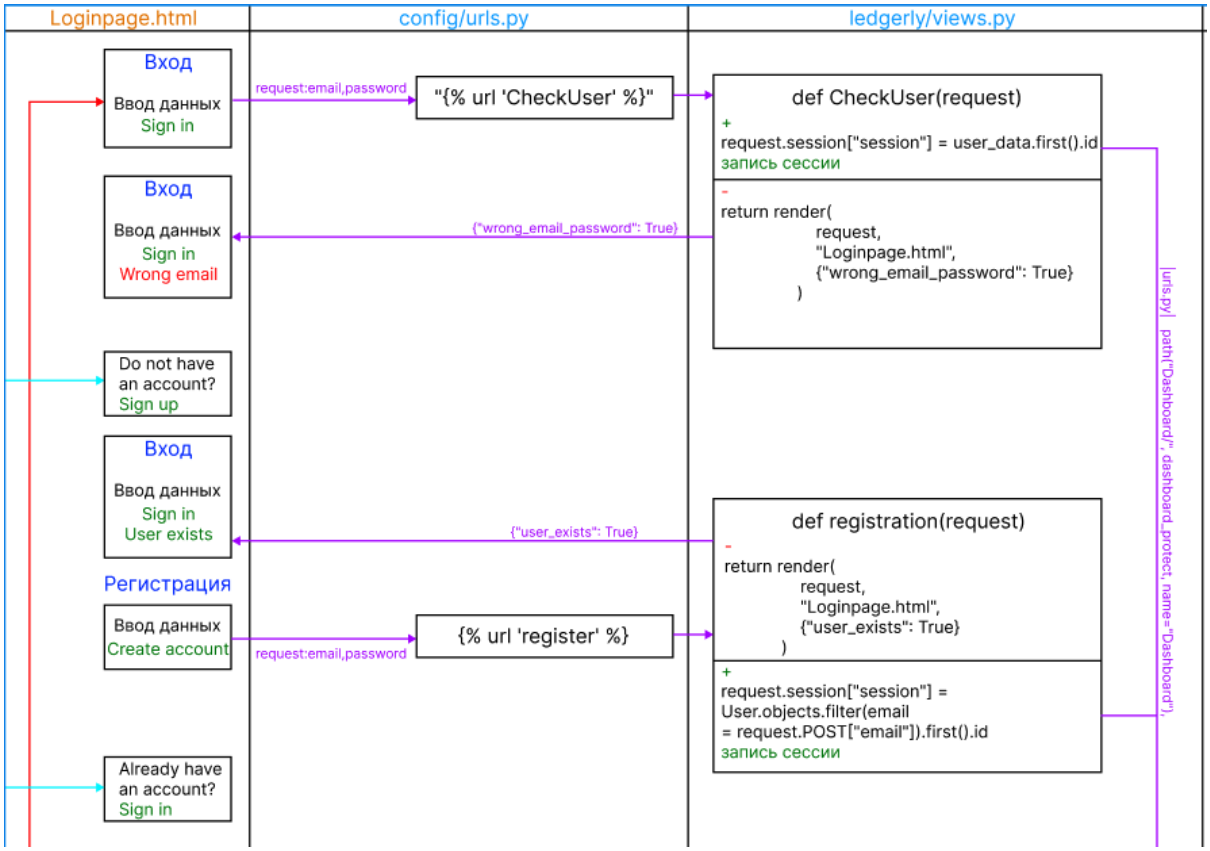
Глава 4. Реализация

4.1 API-маршруты

Метод	Путь	Описание
GET	/	Страница входа (Loginpage.html)
GET	/ Dashboard/	Защищённая панель управления (dashboard_protect)
POST	/ register/	Регистрация нового пользователя (функция registration)
POST	/ CheckUser/	Проверка пользователя (функция CheckUser)
GET	/ logout/	Выход из сессии (функция logout)
GET	/ company/<int:company_id>/price/	Получение цены компании по ID (company_price)
GET	/ company/<int:company_id>/panel/	Панель компании по ID (company_panel)
GET	/ company/<int:company_id>/chart/income/	Данные для графика доходов (по ID компании)
GET	/ company/<int:company_id>/chart/balance/	Данные для графика баланса (по ID компании)
POST	/company/<int:company_id>/chart/cash-flow/	Данные для графика денежных потоков (по ID компании)

- **GET /api/user - получение пользователя**

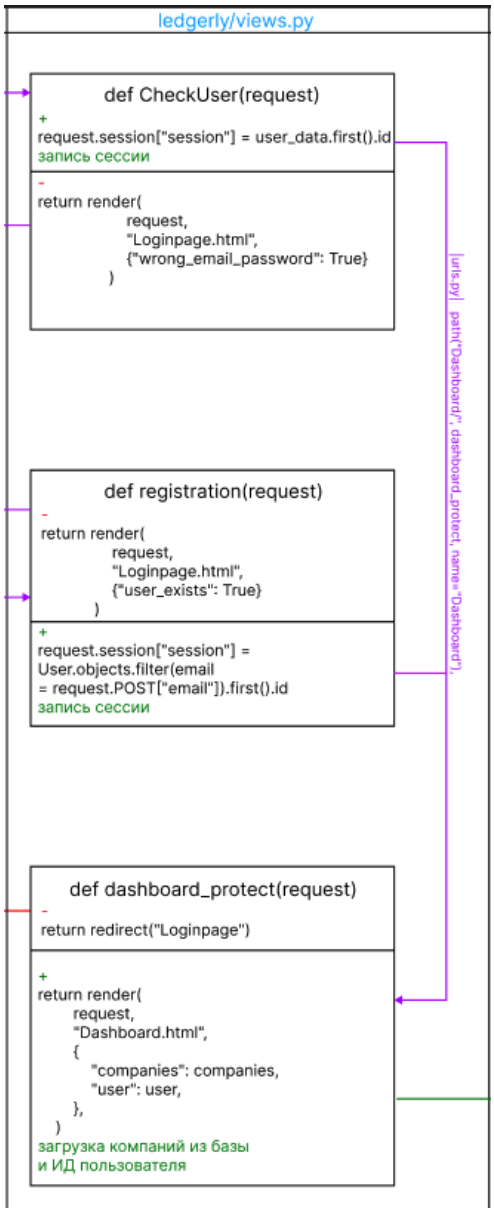
При обращении пользователя к корневому адресу сайта / браузер отправляет GET-запрос на сервер. Django сопоставляет адрес с маршрутом, заданным в файле config/urls.py, и вызывает представление TemplateView, которое отображает шаблон Loginpage.html. Сервер возвращает сформированную HTML-страницу, содержащую формы входа и регистрации. По умолчанию пользователь видит форму входа с полями электронной почты и пароля. При выборе регистрации карточка переключается на форму создания аккаунта без перехода на другую страницу. Сам GET-запрос только загружает интерфейс: он не проверяет учётные данные и не создаёт пользователя. После заполнения и отправки формы браузер выполняет отдельный POST-запрос: для входа — на /CheckUser/, для регистрации — на /register/.



- **POST /register/ - Регистрация нового пользователя (функция registration)**

При отправке формы регистрации браузер выполняет POST-запрос на маршрут /Register/, передавая введённые пользователем данные. Django сопоставляет адрес с маршрутом и вызывает функцию registration, которая получает данные из request.POST и проверяет заполнение обязательных полей, а также наличие пользователя с указанной

электронной почтой. Если обязательные данные отсутствуют или email уже используется, функция повторно отображает шаблон Loginpage.html с сообщением об ошибке и открытой стороной регистрации, чтобы пользователь мог исправить ввод. Если проверка пройдена, функция создаёт новую запись пользователя в базе данных, сохраняя пароль в виде хеша, а не открытого текста. После успешной регистрации пользователь перенаправляется на страницу входа, где может авторизоваться с созданными учётными данными.

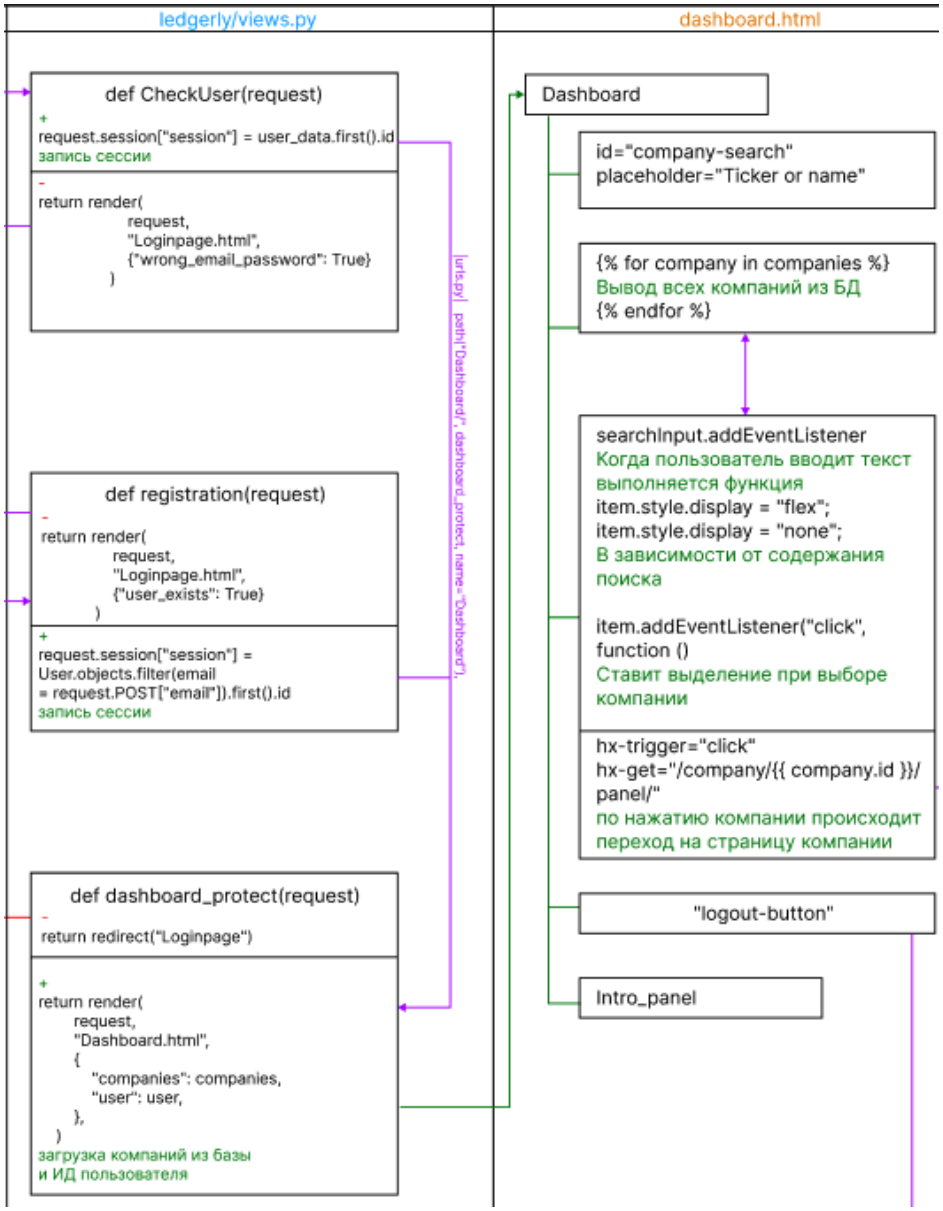


- **POST /CheckUser/ - Проверка пользователя (функция CheckUser)**

Маршрут /CheckUser/ предназначен для обработки формы входа и проверки учётных данных пользователя. После нажатия кнопки входа браузер отправляет email и пароль на сервер, где Django вызывает функцию CheckUser. Функция получает данные из request.POST, проверяет заполнение полей, ищет пользователя с указанным email и проверяет соответствие пароля сохранённому хешу. Если данные верны, идентификатор пользователя сохраняется в сессии, после чего выполняется перенаправление на /Dashboard/. Сессия позволяет приложению распознавать пользователя при последующих запросах. Если поля не заполнены, пользователь не найден или пароль неверный, повторно отображается страница Loginpage.html с сообщением об ошибке.

- **GET /Dashboard/ - Защищённая панель управления (dashboard_protect)**

  Маршрут /Dashboard/ открывает защищённую панель управления и обрабатывается функцией dashboard_protect. При обращении к этому адресу функция проверяет наличие идентификатора пользователя в сессии и соответствующей записи в базе данных. Если пользователь авторизован и его учётная запись существует, сервер отображает страницу панели управления. Если идентификатор отсутствует, пользователь перенаправляется на страницу входа. Если в сессии сохранён идентификатор уже удалённого пользователя, функция очищает недействительную сессию и также перенаправляет на страницу входа. Такая проверка не позволяет получить доступ к панели простым вводом её адреса в браузере и предотвращает ошибку при обращении к несуществующей учётной записи.

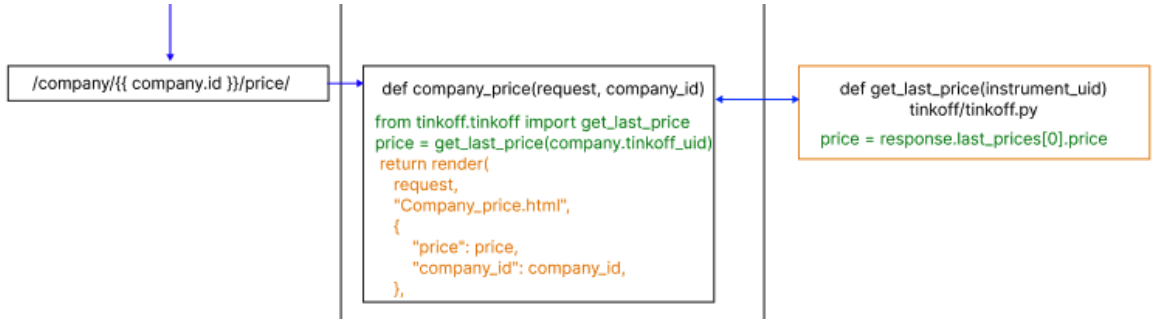


- **GET /Logout/ - Выход из сессии (функция logout)**

Маршрут `/logout/` предназначен для завершения текущей пользовательской сессии и обрабатывается функцией `logout`. При переходе по этому адресу браузер отправляет GET-запрос, после чего функция очищает сессию, удаляя сохранённые в ней данные авторизации. Затем пользователь перенаправляется на страницу входа. При повторном обращении к защищённой панели `/Dashboard/` приложение уже не обнаружит действующую авторизацию и предложит войти снова. При этом учётная запись пользователя и связанные с ней данные в базе сохраняются: выход завершает только текущую сессию, а не удаляет аккаунт.

- **GET / company/<int:company_id>/price/ - Получение цены компании по ID (company_price)**

Маршрут `/company/<int:company_id>/price/` предназначен для получения последней цены финансового инструмента выбранной компании и обрабатывается функцией `company_price`. При получении GET-запроса Django передаёт функции числовой идентификатор компании из адреса. Сначала функция проверяет наличие ключа `"session"` в пользовательской сессии. Если ключ отсутствует, сервер возвращает JSON-сообщение `Unauthorized` со статусом 401. Затем выполняется поиск активной компании в базе данных по переданному идентификатору. Если компания не найдена или неактивна, возвращается JSON-сообщение `Company not found` со статусом 404. Если компания найдена, функция передаёт её идентификатор инструмента `tinkoff_uid` в `get_last_price`, которая получает последнюю цену. Полученное значение вместе с идентификатором компании передаётся в шаблон `Company_price.html`. Сервер возвращает сформированный HTML-ответ для отображения цены в интерфейсе. Таким образом, успешный ответ этого маршрута содержит HTML, а сообщения об отсутствии авторизации или компании возвращаются в формате JSON.



- **GET /company/<int:company_id>/panel/ - Панель компании по ID (company_panel)**

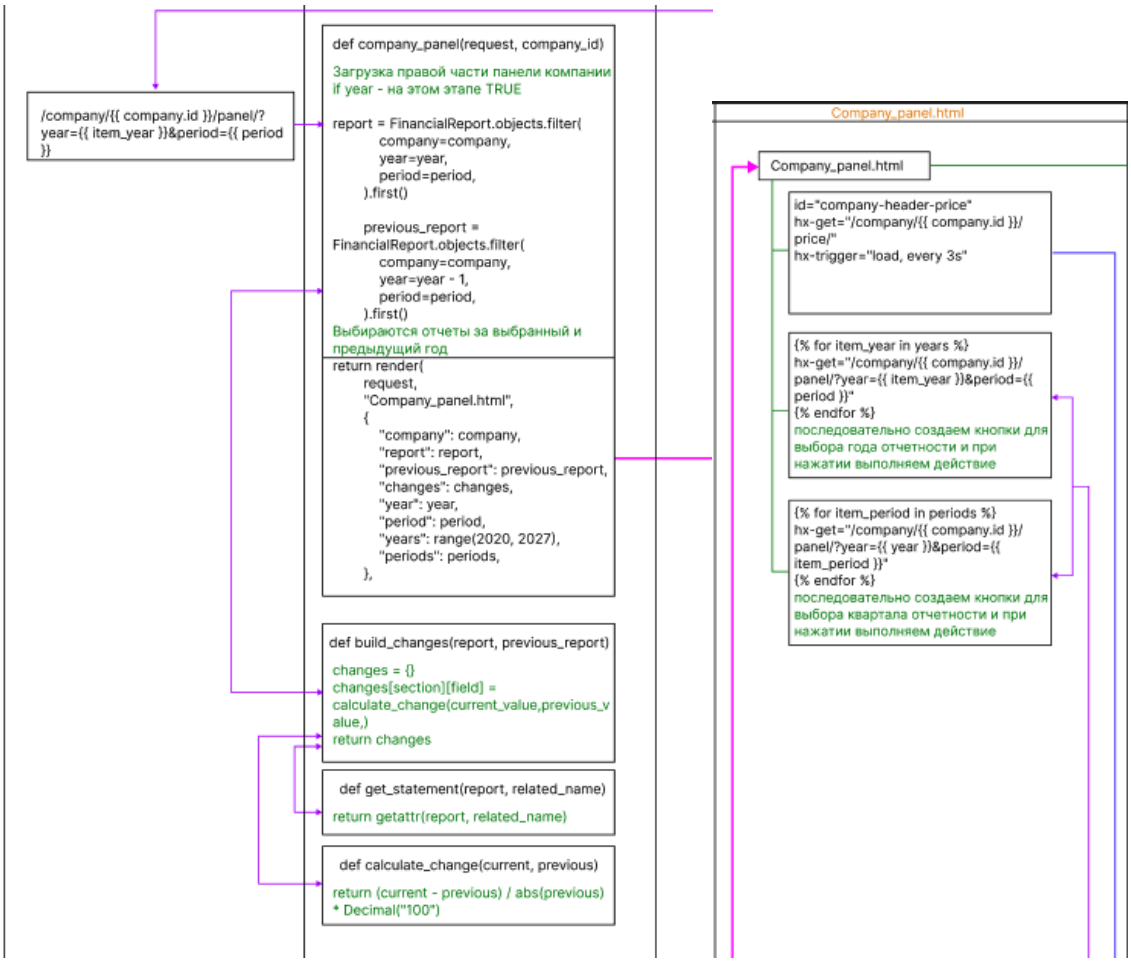
Маршрут `/company/<int:company_id>/panel/` предназначен для отображения финансовой панели выбранной компании и обрабатывается функцией `company_panel`. При получении GET-запроса Django передаёт функции числовой идентификатор компании из адреса. Сначала проверяется наличие ключа `"session"` в пользовательской сессии. Если ключ отсутствует, сервер возвращает JSON-сообщение `Unauthorized` со статусом 401. Затем функция ищет в базе данных активную компанию с указанным идентификатором. Если компания не найдена или неактивна, возвращается JSON-сообщение `Company not found` со статусом 404.

После успешной проверки функция формирует таблицу дивидендов и получает из параметров запроса выбранные год и отчётный период. Если период не указан, используется значение `FY`, обозначающее годовую отчётность. При наличии года из

базы извлекаются финансовый отчёт за выбранный период и отчёт за аналогичный период предыдущего года. На их основе рассчитываются процентные изменения показателей отчёта о прибылях и убытках, бухгалтерского баланса и движения денежных средств. Если данные для сравнения отсутствуют или значение предыдущего года равно нулю, процентное изменение не рассчитывается. Дополнительно формируются графики финансовых показателей и данные прогнозов аналитиков. Параметр `chart_mode` определяет режим графика отчёта о прибылях и убытках, а графики баланса и денежных потоков формируются в годовом режиме.

Подготовленные данные передаются в шаблон `Company_panel.html`, из которого сервер формирует HTML-ответ. В интерфейсе Dashboard этот маршрут вызывается с помощью HTMX при нажатии на компанию в боковом списке. Полученный HTML заменяет содержимое блока `company-content`, поэтому панель выбранной компании

открывается без полной перезагрузки страницы.



- **GET / company/<int:company_id>/chart/income/ - Данные для графика доходов (по ID компании)**